Crypto DWH Leitfaden fuer Data Engineers

Crypto DWH Leitfaden fuer Data Engineers

Datenfluss, Verarbeitung, Deployment pro Schritt

Trainingsfolien fuer Data-Engineering-Auszubildende

- Datum: 2026-02-26
- Fokus: Von Rohdaten bis Data Mart und Dashboard

Lernziele

Nach dieser Session koennt ihr:

1. den End-to-End Datenfluss der Pipeline erklaeren.
2. pro Schritt Zweck, Input, Output und Deployment verstehen.
3. `OLTP` und `OLAP` in dieser Architektur korrekt einordnen.
4. `Star`, `Snowflake`, `Galaxy` und `Data Mart` am Projektbeispiel unterscheiden.

Agenda

1. Problem und Zielbild
2. Pipeline-Schritte mit Deployment-Diagrammen
3. ETL, OLTP, OLAP im Projektkontext
4. Data Mart + Schema-Patterns (Star/Snowflake/Galaxy)
5. Operativer Leitfaden fuer den taeglichen Betrieb

Problem und Zielbild

Problem

- Gleiche Coins haben je Exchange unterschiedliche Preise.
- Plattformqualitaet (Latency, Update-Frequenz, Disconnects) schwankt.
- Ohne saubere Zeitausrichtung sind Vergleiche nicht fair.

Zielbild

- Nachvollziehbare DWH-Pipeline von Ingestion bis Mart.
- Vergleichbare KPI je Symbol und Exchange.
- Dashboard als schneller Analysezugang.

End-to-End Datenfluss (Uebersicht)

ETL/ELT in diesem Projekt

Praktische Sicht

- **E (Extract):** WebSocket-Ticks und Connection-Events aus Exchanges.
- **L (Load):** Zuerst in Raw-DBs (OLTP-nah, write-optimiert).
- **T (Transform):** In Staging/Cleansing/Core/Marts fuer Analyse.

Warum so?

- Online-Ingestion bleibt stabil.
- Batch-Transformationen koennen reproduzierbar laufen.

OLTP vs OLAP im Projekt

OLTP-Zone (schreibintensiv)

- `worker_*_crypto_ws_ticks.db`
- viele Inserts, laufender Betrieb, technische Rohdaten

OLAP-Zone (lese- und analyseintensiv)

- `cleaned_*.db`, `core_kpi.db`, Mart-Views/Cache
- aggregierte KPI, zeitliche Vergleiche, Dashboard-Abfragen

Schritt 1: Ingestion (Extract + Raw Load)

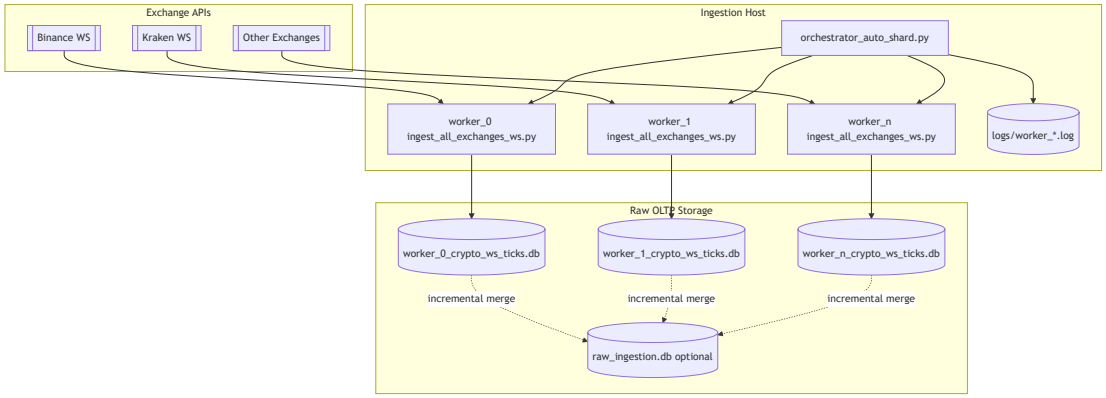
Verarbeitung

- Orchestrator shardet Exchanges auf Worker-Prozesse.
- Worker schreiben Ticks und Events kontinuierlich in SQLite.
- Restart- und Supervision-Logik stabilisiert den 24/7-Betrieb.

Ergebnis

- Rohdaten mit Ingestion-Timestamp als Basis fuer spaetere KPI.

Deployment


Exchange APIs
Binance WS
Kraken WS
Other Exchanges
Ingestion Host
orchestrator_auto_shard.py
worker_0
ingest_all_exchanges_ws.py
worker_1
ingest_all_exchanges_ws.py
worker_n
ingest_all_exchanges_ws.py
logs/worker_*.log
Raw OLTP Storage
worker_0_crypto_ws_ticks.db
worker_1_crypto_ws_ticks.db
worker_n_crypto_ws_ticks.db
incremental merge
incremental merge
incremental merge
raw_ingestion.db optional

Schritt 2: Staging (Entkopplung)

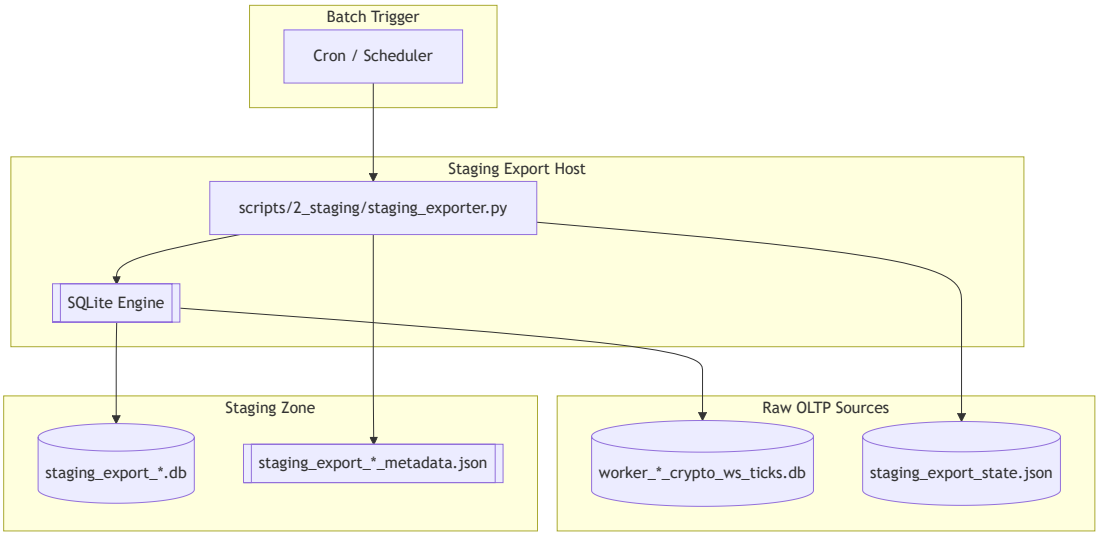
Verarbeitung

- Geplanter Job exportiert ein Zeitfenster (z. B. letzte 24h).
- Optional inkrementell ueber Watermark-State.
- Lauf erzeugt Datenexport plus Metadaten-JSON.

Ergebnis

- Stabile Input-Scheibe fuer Cleansing und Core.

Deployment


Batch Trigger
Cron / Scheduler
Staging Export Host
scripts/2_staging/staging_exporter.py
SQLite Engine
Staging Zone
staging_export_*.db
staging_export_*_metadata.json
Raw OLTP Sources
worker_*_crypto_ws_ticks.db
staging_export_state.json

Schritt 3: Cleansing (Zeitliche Vergleichbarkeit)

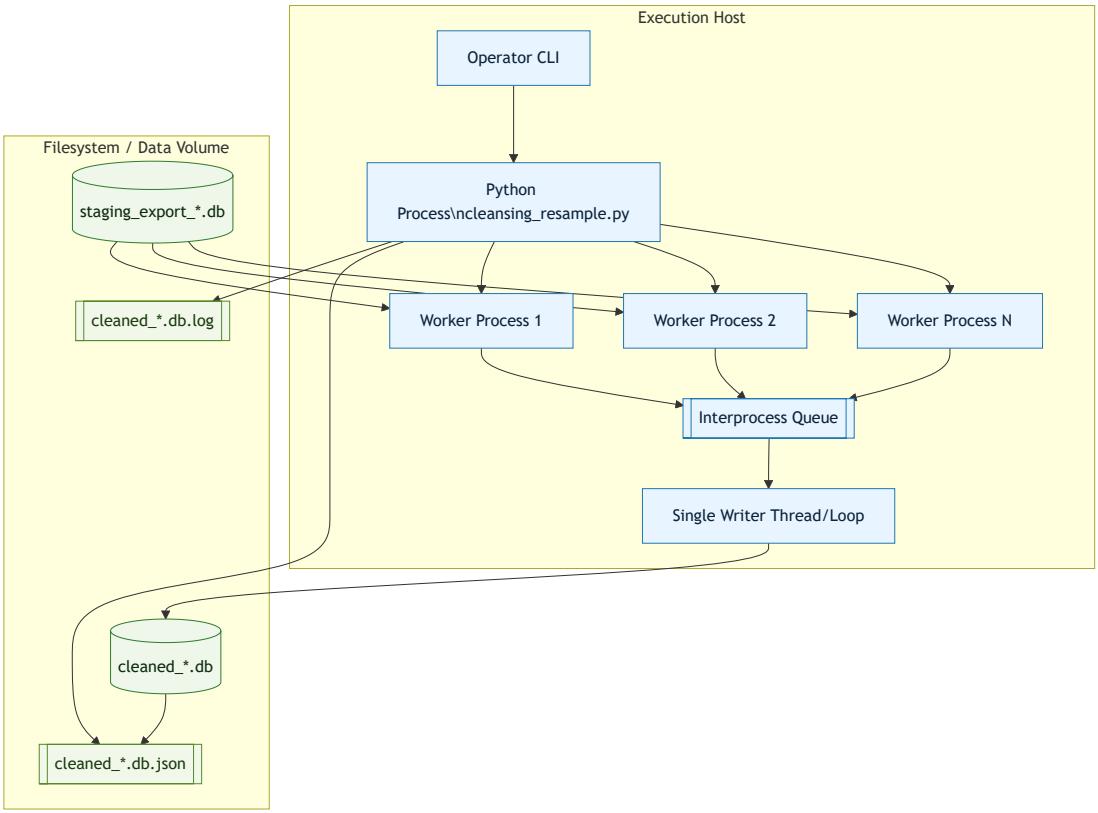
Verarbeitung

- Resampling auf ein einheitliches Raster (z. B. 60s Bins).
- Fill-Strategien: `observed`, `forward_fill`, optional `interpolation`.
- Qualitaetsflags markieren `missing`, `stale`, Alter der Werte.

Ergebnis

- Vergleichbare Zeitreihen pro `(exchange_id, symbol)`.

Deployment


Execution Host
Operator CLI
Python Process\ncleansing_resample.py
Worker Process 1
Worker Process 2
Worker Process N
Interprocess Queue
Single Writer Thread/Loop
Filesystem / Data Volume
staging_export_*.db
cleaned_*.db.log
cleaned_*.db
cleaned_*.db.json

Schritt 4: Core (KPI-Berechnung, OLAP Kern)

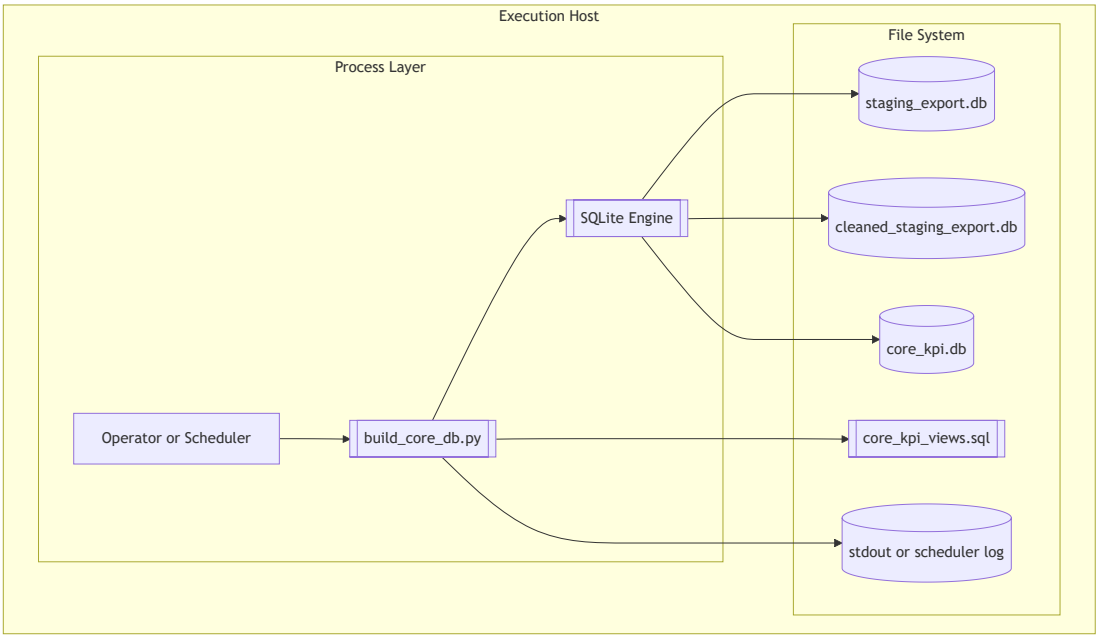
Verarbeitung

- KPI-Views fuer:
  - `latency_ms`
  - `update_frequency_hz`
  - `disconnect_count`
  - `price_diff_abs` / `price_diff_pct`
- SQL Assertions validieren Datenqualitaet und Vollstaendigkeit.

Ergebnis

- Konsistente KPI-Schicht als Single Source of Truth.

Deployment


Execution Host
File System
Process Layer
staging_export.db
SQLite Engine
cleaned_staging_export.db
core_kpi.db
Operator or Scheduler
build_core_db.py
core_kpi_views.sql
stdout or scheduler log

Schritt 5: Marts (Analyse-Verbrauchsschicht)

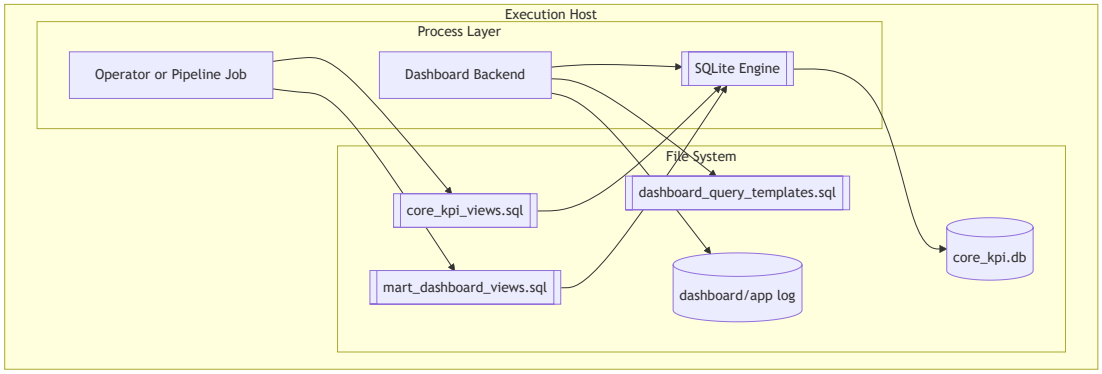
Verarbeitung

- Mart-Views formen KPI fuer konkrete Analysefragen um.
- Dashboard liest nur konsumfertige Views/Cache-Tabellen.
- Keine KPI-Neuberechnung in der UI.

Ergebnis

- Schnelle, konsistente Auswertung fuer Fachnutzer.

Deployment


Execution Host
Process Layer
Operator or Pipeline Job
Dashboard Backend
SQLite Engine
File System
core_kpi_views.sql
dashboard_query_templates.sql
mart_dashboard_views.sql
dashboard/app log
core_kpi.db

Data Mart im Projekt

Was ist der Data Mart hier?

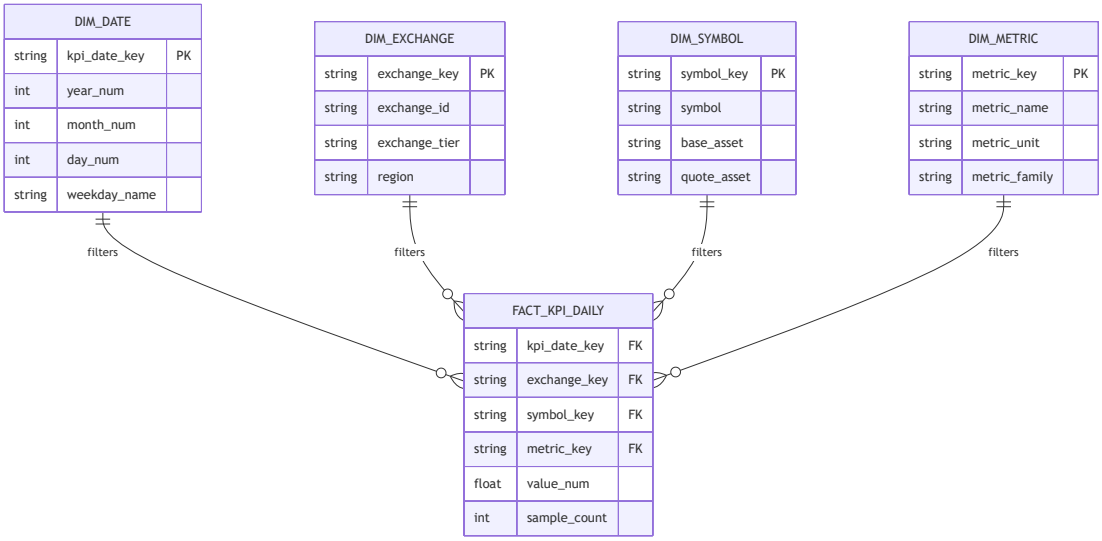
- Themenbezogener Ausschnitt fuer Dashboard-Konsum.
- Fokus: Preisabweichung und Plattformqualitaet.

Konkrete Objekte

- `vw_mart_dashboard_platform_quality_daily`
- `vw_mart_dashboard_price_deviation_daily`
- `vw_mart_dashboard_price_curve_24h_binance`

Star Schema (einfach, schnell konsumierbar)

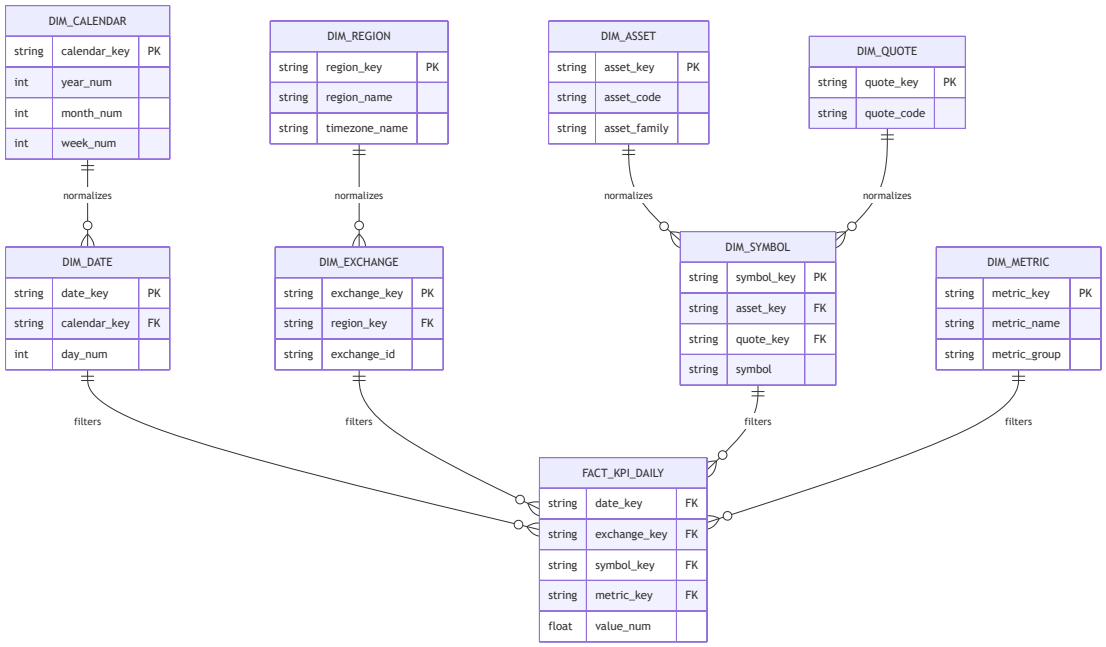
Empfehlung: - fuer stabile Dashboard-KPI mit klaren Dimensionen.


DIM_DATE
string kpi_date_key PK
int year_num
int month_num
int day_num
string weekday_name
DIM_EXCHANGE
string exchange_key PK
string exchange_id
string exchange_tier
string region
DIM_SYMBOL
string symbol_key PK
string symbol
string base_asset
string quote_asset
DIM_METRIC
string metric_key PK
string metric_name
string metric_unit
string metric_family
filters
filters
filters
filters
FACT_KPI_DAILY
string kpi_date_key FK
string exchange_key FK
string symbol_key FK
string metric_key FK
float value_num
int sample_count

Snowflake Schema (staerker normalisiert)

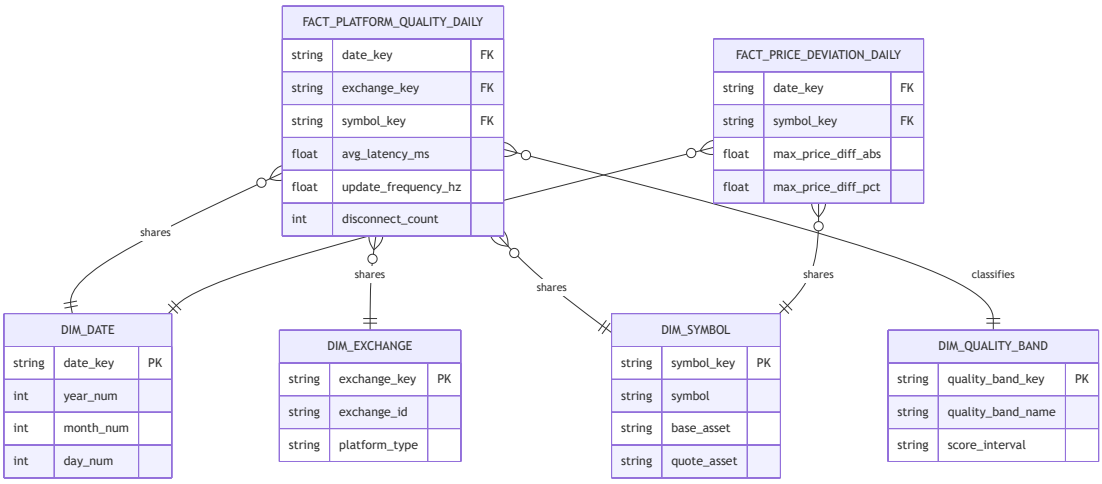
Empfehlung: - wenn Dimensionen wachsen und Redundanz gesenkt werden soll.

Trade-off: - weniger Duplikate, aber komplexere Joins.


DIM_CALENDAR
string calendar_key PK
int year_num
int month_num
int week_num
DIM_REGION
string region_key PK
string region_name
string timezone_name
DIM_ASSET
string asset_key PK
string asset_code
string asset_family
DIM_QUOTE
string quote_key PK
string quote_code
normalizes
normalizes
normalizes
normalizes
DIM_DATE
string date_key PK
string calendar_key FK
int day_num
DIM_EXCHANGE
string exchange_key PK
string region_key FK
string exchange_id
DIM_SYMBOL
string symbol_key PK
string asset_key FK
string quote_key FK
string symbol
DIM_METRIC
string metric_key PK
string metric_name
string metric_group
filters
filters
filters
filters
FACT_KPI_DAILY
string date_key FK
string exchange_key FK
string symbol_key FK
string metric_key FK
float value_num

Galaxy Schema (mehrere Facts, geteilte Dimensionen)

Empfehlung: - wenn mehrere Analysebereiche parallel laufen. - hier: Plattformqualitaet und Preisabweichung als getrennte Facts.


FACT_PLATFORM_QUALITY_DAILY
string date_key FK
string exchange_key FK
string symbol_key FK
float avg_latency_ms
float update_frequency_hz
int disconnect_count
FACT_PRICE_DEVIATION_DAILY
string date_key FK
string symbol_key FK
float max_price_diff_abs
float max_price_diff_pct
shares
shares
shares
shares
classifies
DIM_DATE
string date_key PK
int year_num
int month_num
int day_num
DIM_EXCHANGE
string exchange_key PK
string exchange_id
string platform_type
DIM_SYMBOL
string symbol_key PK
string symbol
string base_asset
string quote_asset
DIM_QUALITY_BAND
string quality_band_key PK
string quality_band_name
string score_interval

Wann welches Schema?

1. **Star**: Startpunkt fuer MVP-Dashboard, einfache SQL, schnelle Lieferung.
2. **Snowflake**: bei vielen Dimension-Attributen und Governance-Fokus.
3. **Galaxy**: bei mehreren Fact-Domaenen mit gemeinsamen Dimensionen.

Praxisempfehlung: - MVP mit Star starten, bei Bedarf zu Snowflake/Galaxy evolvieren.

Operativer Leitfaden (taeglicher Ablauf)

1. Ingestion laeuft kontinuierlich (Health und Disconnects beobachten).
2. Staging exportiert den Zielzeitraum.
3. Cleansing erzeugt zeitlich ausgerichtete Serien.
4. Core berechnet/validiert KPI.
5. Mart-Views und Cache refreshen.
6. Dashboard fuer Analyse freigeben.

Qualitaets-Checkliste fuer Auszubildende

- Sind alle Zeitstempel in UTC?
- Ist Symbol-Mapping exchange-uebergreifend konsistent?
- Sind Missing/Stale-Werte sichtbar markiert?
- Ist die Definition von "Connection Drop" dokumentiert?
- Werden KPI nur im SQL-Layer gerechnet (nicht in UI)?

PDF-Erzeugung aus Markdown

Beispiel mit Marp CLI:

```
npx @marp-team/marp-cli docs/7_presentation/crypto_dwh_data_engineering_leitfaden_slides.md
```

Die Datei ist Marp-kompatibel und fuer direkten PDF-Export vorbereitet.